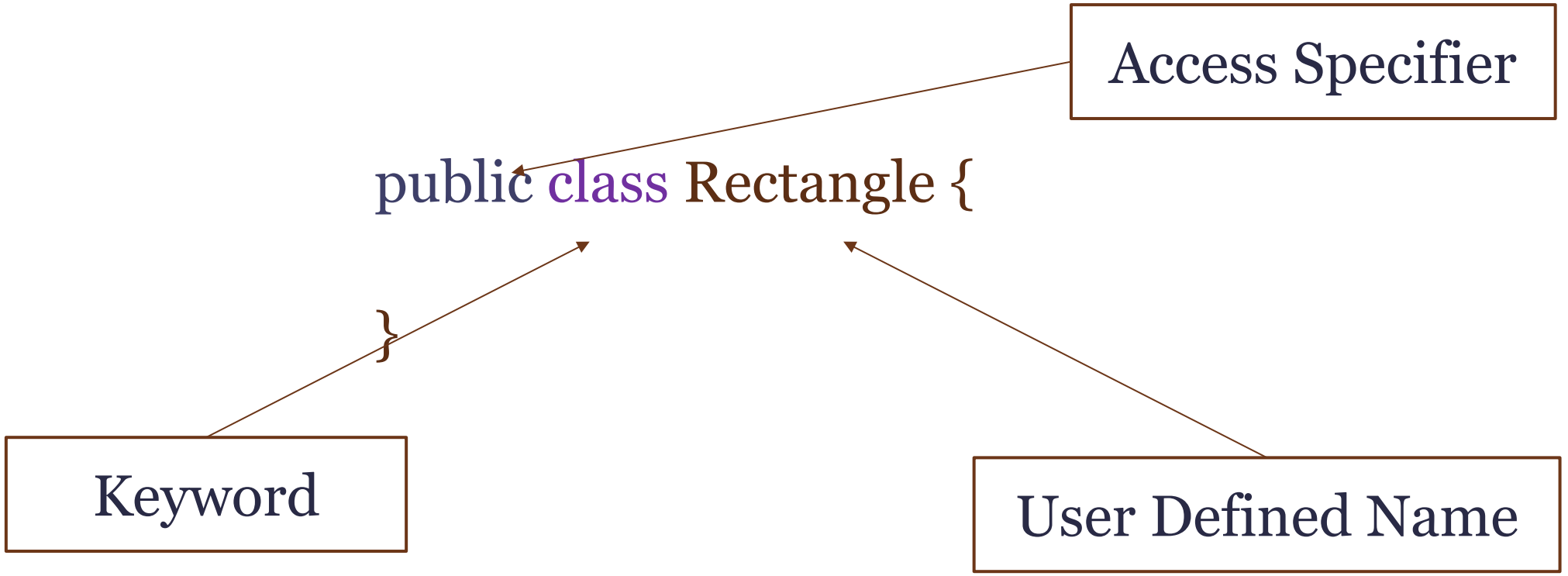
Object Oriented Programming

Java

Class Declaration



Ways to Create Objects

- There are **5 different ways** to create objects in java:
  - Using ‘new’ keyword
  - Using Class.forName()
  - Using clone()
  - Using Object Deserialization
  - Using newInstance() method

Above three ways will be discussed.

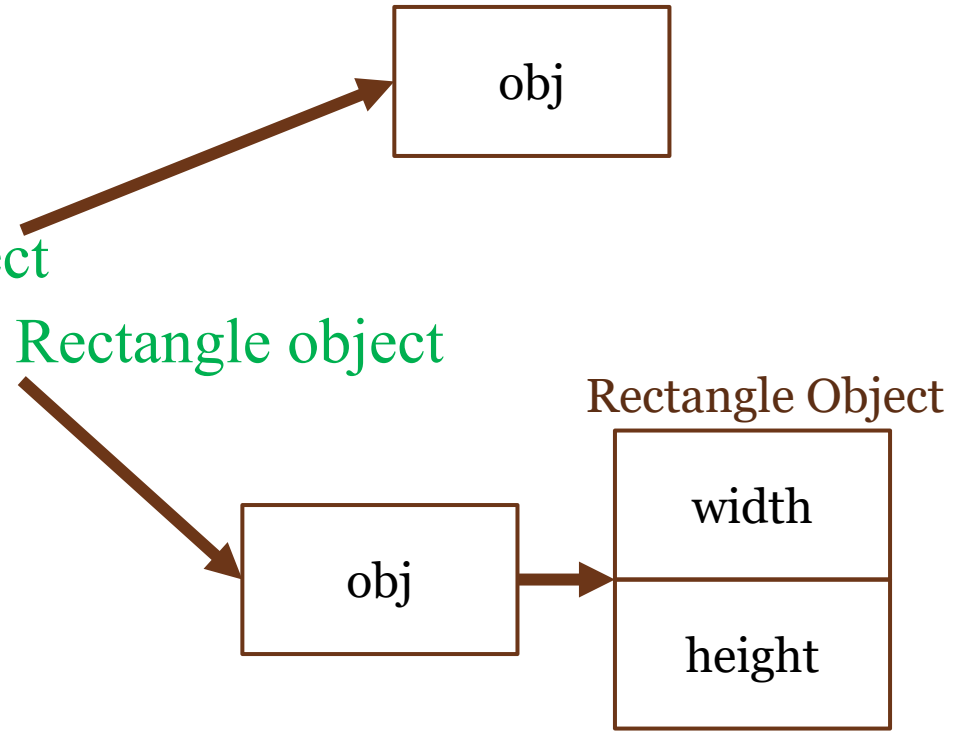
Ways to Create Objects

- **Using 'new' keyword:**

```
Rectangle obj = new Rectangle();
```

It can be rewritten like this:

```
Rectangle obj; // declare reference to object
obj = new Rectangle();          // allocate a Rectangle object
```



Ways to Create Objects

- **Using Class.forName():**

  Class exampleClass = Class.forName("package.Rectangle");
  Object obj = exampleClass.newInstance();

- **Using clone():**

  The clone() can be used to create a copy of an existing object.
  Rectangle object = (Rectangle)obj.clone();

Anonymous Object

- If you have to use an object **only once**, anonymous object is a good approach.
- The anonymous object is created and dies instantaneously.
- Anonymous object initialization:

```
new Rectangle()
```

Anonymous Object Example

```
public class Rectangle {
        void area(int w, int h){
                int a = w*h;
                System.out.println(“ The area is ”+a);
        }
        public static void main(String[] args){
                new Rectangle().area(5, 6);
        }
}
```

Access Modifiers

- **Two types of modifiers:**
  - Access Modifiers
  - Non-access Modifiers
- **4 types of access modifiers:**
  - **public:**
    - ⓧ Visible to the world
  - **private:**
    - ⓧ Variables or methods declared with access modifier *private* are accessible only to methods of the class in which they're declared.
    - ⓧ Declaring instance variables with access modifier *private* is known as **data hiding** or **information hiding**.

Access Modifiers

- **protected:**
  - ⓧVisible to the packages and all subclasses.
- **default:**
  - ⓧVisible to the default package. No modifiers are needed.

Variable Types

- Local variables
- Instance variables
- Class/Static variables

Local Variables

- Local variables are declared in method, constructors or blocks.
- Access modifiers cannot be used for local variables.
- They are visible only within the declared method, constructor or block.

```
public class Rectangle {
        void area(int w, int h){
                int a = w*h;
                System.out.println(" The area is "+a);
        }
        public static void main(String[] args){
                new Rectangle().area(5, 6);
        }
}
```

a is the local variable

Instance Variables

- Instance variables are declared in a class, but outside a method, constructor or block.
- When a space is allocated for an object in the heap, a slot for each instance variable value is created.
- Access modifiers can be given for instance variables.
- Instance variables have default values.

Instance Variables

```
public class Rectangle {
        private double width;
        private double length;
        void area(){
        }
        public static void main(String [] args){
                Rectangle obj = new Rectangle();
                obj.width = 10;
                obj.length = 20;
        }
}
```

Instance variable

Using dot operator instance variable can be accessed

Class/Static Variables

- Class/Static variables are declared with the *static* keyword in a class, but outside a method, constructor or block.
- There would be **only one copy** of each class variable per class regardless of how many objects are created from it.
- Static variables are rarely used other than being declared as *final* and as constants.
- Static variables are created when the program starts and destroyed when the program stops.
- Most static variables are declared as *public* since they must be available for users of the class.
- Static variables can be accessed by calling with the class name *ClassName.VariableName*.

Class/Static Variables

```
public class Car{
        private static double price;
        public static final String manufacturer = "Japan Co.";
        void speed(){
        }
        public static void main(String[] args){
                price = 23000;
                System.out.println("Manufacturer :"+ manufacturer);
        }
}
```

Static variables can be accessed from an outside class as *Car.manufacturer*

Constructor in Java

- Constructor in java is a special type of method that is used to initialize the object.
- Java constructor is invoked at the time of object creation. It constructs the values i.e. provides data for the object that is why it is known as constructor.
- There are basically **two rules** defined for the constructor.
  - Constructor name must be **same** as its class name
  - Constructor must have **no explicit return type**

Constructor in Java

- Constructor cannot be called like methods.
- Constructors are called automatically as soon as object gets created.
- Constructor can accept parameter.
- Default constructor automatically called when object is created.

Constructor in Java

Java Default Constructor:

A constructor that has no parameter is known as default constructor. For example:

```
ClassName(){
        ……
}
```

- If there is no constructor in a class, compiler automatically creates a default constructor.

Default Constructor

```
public class Rectangle{
        private double width;
        private double length;
        public Rectangle(double w, double l){
                width = w;
                length = l;
        }
        public static void main(String[] args){
                Rectangle obj = new Rectangle();
        }
}
```

Note: when no constructor is provided, compiler defines a default one. However when a constructor is provided, compiler doesn't create a default one.

Constructor Overloading

- Constructor can be overloaded passing different number of arguments.

```
public class Rectangle{
        private double width;
        private double length;
        public Rectangle(double a){
                width = a;
                length = a;
        }
        public Rectangle(double w, double l){
                width = w;
                length = l;
        }
}
```

```
public static void main(String[] args){

        Rectangle obj = new Rectangle(6);

        Rectangle obj = new Rectangle(10, 8);

}
```

Difference between Constructor and Method

Java Constructor	Java Method
Constructor is used to initialize the state of an object.	Method is used to expose behavior of an object.
Constructor must not have return type.	Method must have return type.
Constructor is invoked implicitly.	Method is invoked explicitly.
The java compiler provides a default constructor if you don't have any constructor.	Method is not provided by compiler in any case.
Constructor name must be same as the class name.	Method name may or may not be same as class name.

Important Notes

Q) Does constructor return any value?

Answer: yes, that is current class instance (You cannot use return type yet it returns a value).

Q) Can constructor perform other tasks instead of initialization?

Answer: yes, like object creation, starting a thread, calling method etc. You can perform any operation in the constructor as you perform in the method.